Async in JavaScript

Shoaib Ghachi

MCT | Technical Trainer | Mentor |SME

Overview

-Introduction to Asynchronous Programming

-Why Do We Need Async?

-What is Callbacks?

-What is Promise? (ES6)

-What is Async/Await? (ES8)

-Why Async/Await is Better?

-Comparison Table

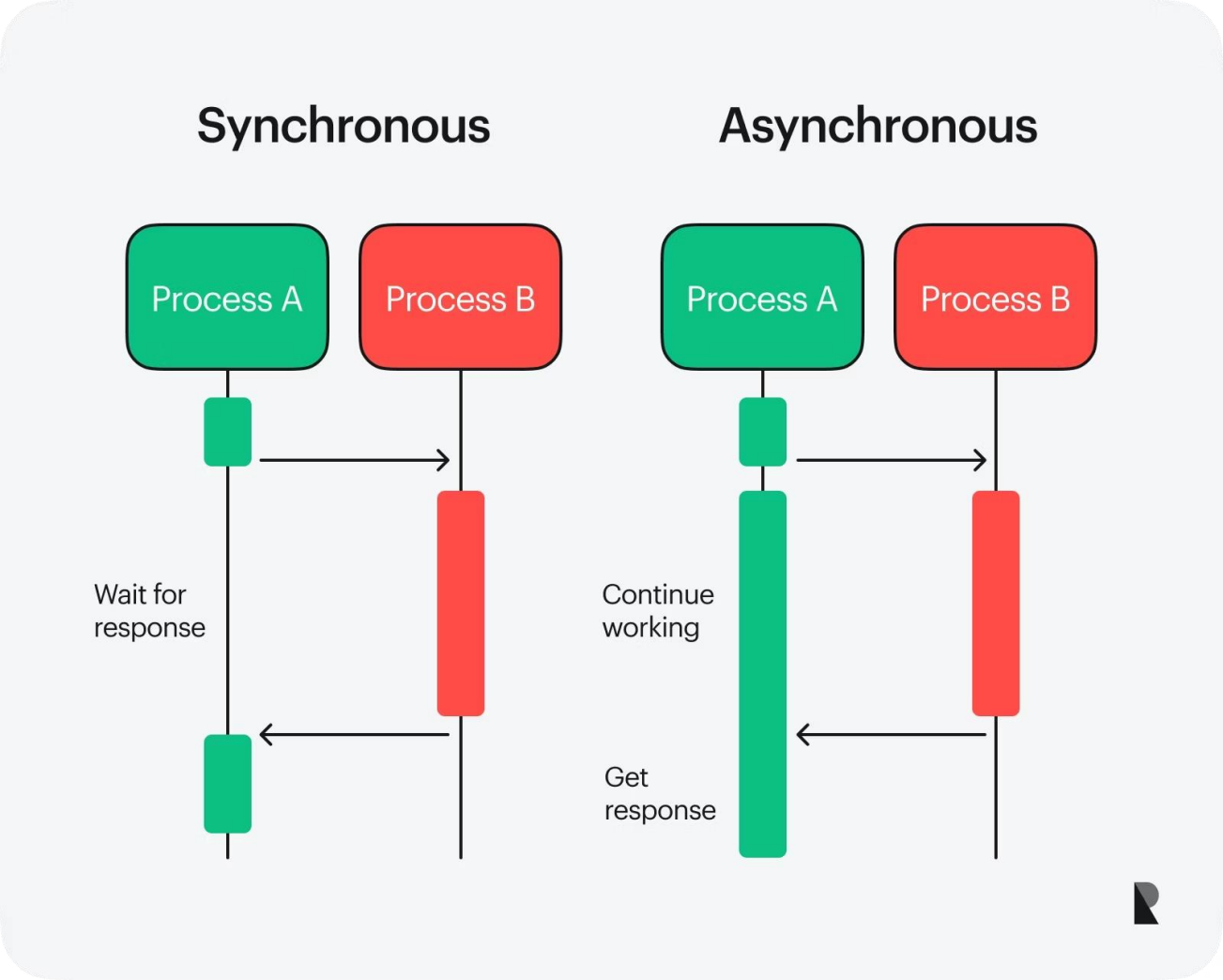
Introduction to Asynchronous Programming

📌 What is Asynchronous Programming?

- JavaScript is **Single-threaded**
- Can perform **non-blocking operations**
- Executes long-running tasks in background
- Uses:
  - Callbacks
  - Promises
  - Async/Await

- **Async Evolution:**

Callback → Promise → Async/Await



Async means "Don't block execution. Continue other work while waiting."

Why Do We Need Async?

📌 Real-world scenarios:

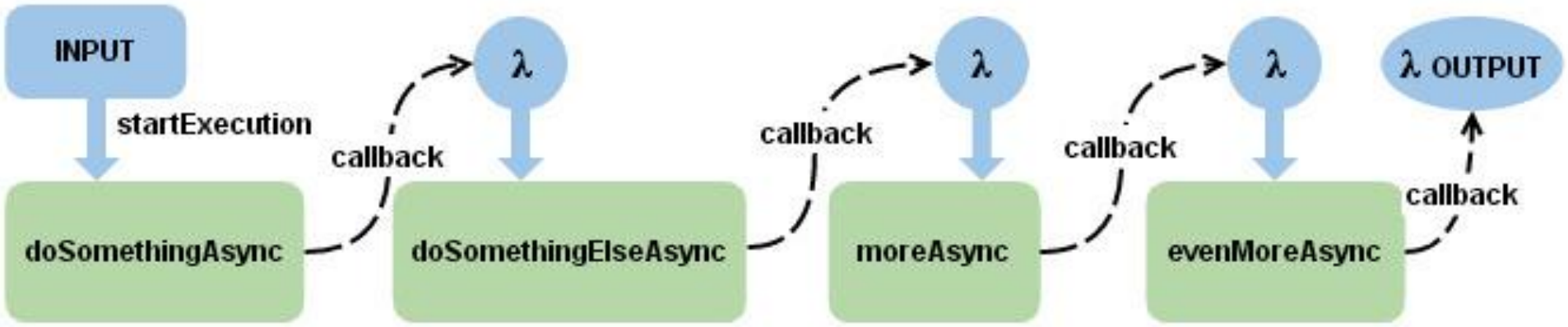
- Fetching data from server
- API calls
- Reading files
- Database operations
- setTimeout / setInterval

What is Callback?

📌 Definition:

A function passed as an argument to another function.

//Callback nesting leads to Callback Hell.



Problem with Callbacks

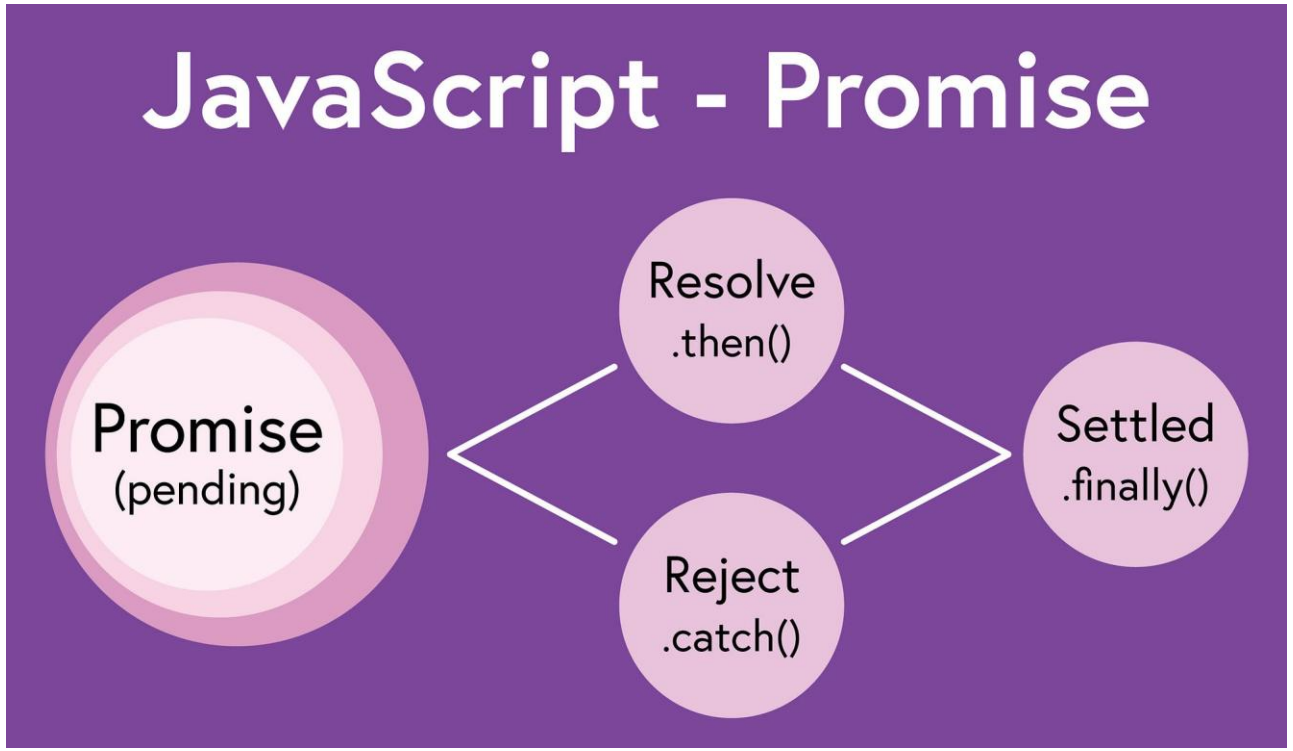
- ❌ Hard to read
- ❌ Difficult error handling

What is Promise?

📌 Definition:

A Promise represents a value that may be:

- Resolved (Success)
- Rejected (Failure)
- Pending



Create → Pending → Resolved / Rejected

Async/Await

📌 Modern way to handle Promises

- `async` makes function return a Promise
- `await` waits for Promise to resolve

- **Why Async/Await is Better?**

  ✅ Cleaner syntax

  ✅ Looks like synchronous code

  ✅ Easy error handling with try/catch

  ✅ Avoids callback hell

Comparison Table

Feature	Callback	Promise	Async/Await
Readability	Low	Medium	High
Error Handling	Difficult	.catch()	try/catch
Nested Code	High	Less	No
Modern Usage	Old	ES6	ES8+